

Atelier 6 : RabbitMQ sur AlwaysData (microservices)

1- Introduction

Ce projet implémente une **architecture microservices basée sur Node.js** pour une application de gestion asynchrone de commandes. Il démontre les principes de découplage et de communication asynchrone entre plusieurs services indépendants via un système de files de messages RabbitMQ.

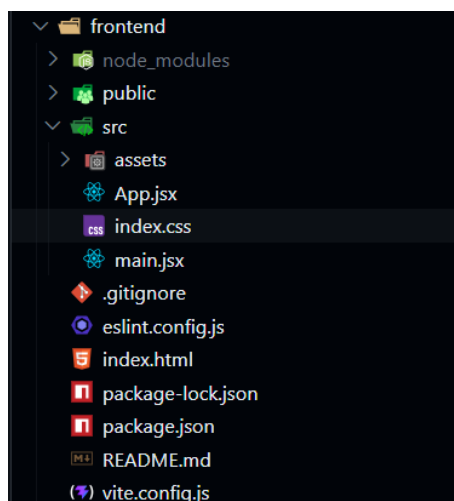
2- Mise en place sur AlwaysData (RabbitMQ)



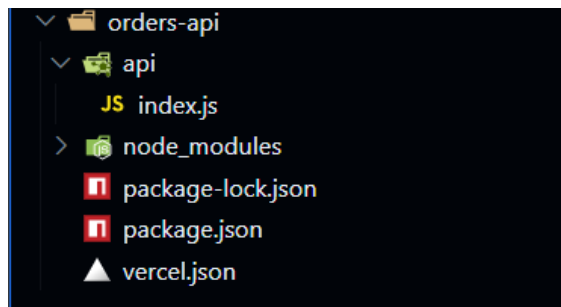
3- Architecture Générale

Le projet est composé de quatre services principaux :

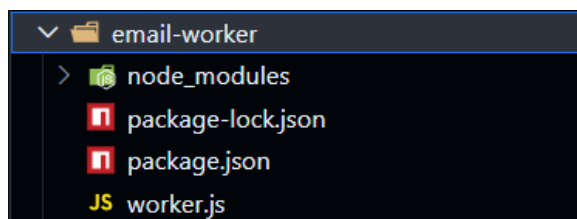
1. Frontend — Interface utilisateur (Vite + Vue/React) où les clients placent des commandes



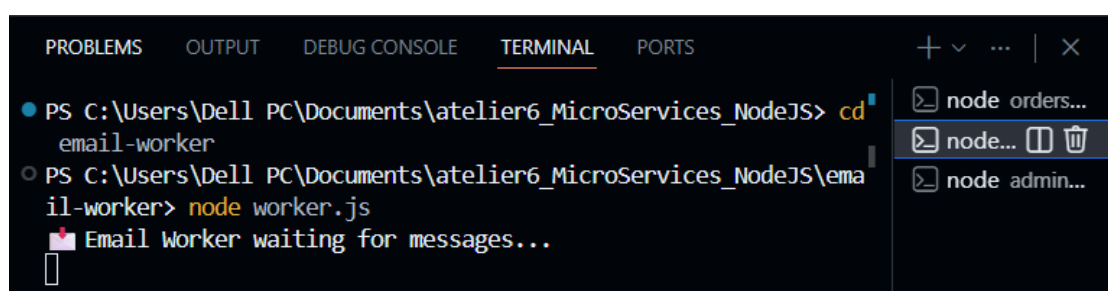
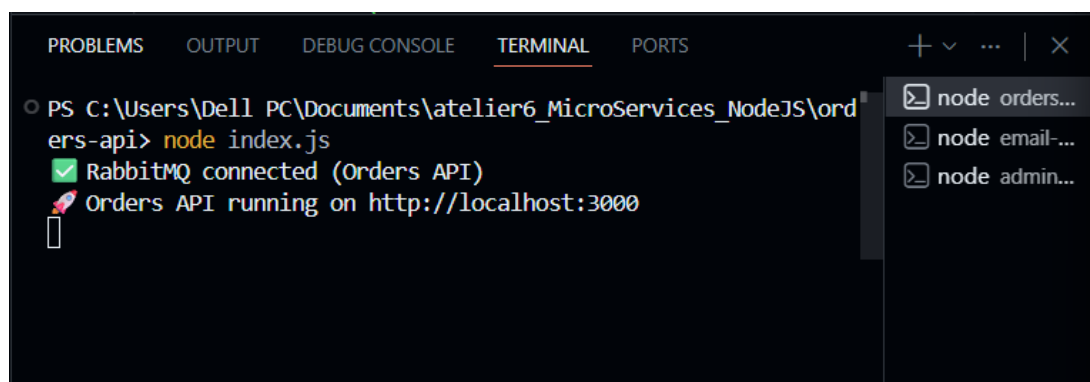
2. Orders API — API REST qui expose les endpoints pour créer et consulter les commandes, puis publie les événements vers RabbitMQ



3. Email Worker — Service consommateur qui reçoit les événements de nouvelles commandes et envoie automatiquement des e-mails de confirmation



3- Tests locaux



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS + v ... | x

PS C:\Users\Dell PC\Documents\atelier6_MicroServices_NodeJS> cd
● admin-ui
○ PS C:\Users\Dell PC\Documents\atelier6_MicroServices_NodeJS\admin-ui> node index.js
  ✓ Admin UI connected to RabbitMQ
  🚀 Admin UI on http://localhost:4000
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS + v ... | x

PS C:\Users\Dell PC\Documents\atelier6_MicroServices_NodeJS> cd
● email-worker
○ PS C:\Users\Dell PC\Documents\atelier6_MicroServices_NodeJS\email-worker> node worker.js
  📧 Email Worker waiting for messages...
  📧 Sending email (simulated) to: mohamedsaidbenyaiche@gmail.com
  ✓ Email sent for order: 177249200573
  📧 Sending email (simulated) to: mohamedsaidbenyaiche@gmail.com
  ✓ Email sent for order: 1772492128225
  📧 Sending email (simulated) to: mohamedsaidbenyaiche@gmail.com
  ✓ Email sent for order: 1772492252206
```

POST Post data x + No environment v

My Collection / Post data Save v Share

POST http://localhost:3000/orders Send v

Docs Params Authorization Headers (8) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```
1 {
2   "email": "test@mail.com",
3   "total": 120
4 }
```

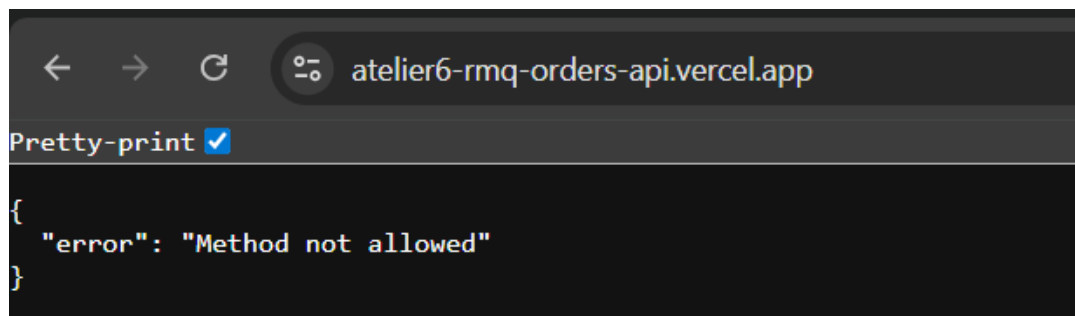
Body v ↺ 201 Created • 117 ms • 413 B • Save Response ...

{ JSON v Preview Visualize v

```
1 {
2   "message": "Order created. Email will be sent asynchronously.",
3   "order": {
4     "orderId": 1772491253919,
5     "email": "test@mail.com",
6     "total": 120,
7     "createdAt": "2026-03-02T22:40:53.919Z"
8   }
9 }
```

4- Service 1 : Orders API (Producteur)

<https://atelier6-rmq-orders-api.vercel.app/>



```
JS index.js ...api X ▲ vercel.json App.jsx index.css JS index.js admin-ui ...
orders-api > api > JS index.js > ...
1  const amqp = require("amqplib");
2  const cors = require("cors");
3
4  let channel;
5
6  const USERNAME = process.env.RABBIT_USER;
7  const PASSWORD = process.env.RABBIT_PASS;
8  const HOST = process.env.RABBIT_HOST;
9  const VHOST = process.env.RABBIT_VHOST;
10
11 const RABBIT_URL = `amqp://${USERNAME}:${PASSWORD}@${HOST}:5672/${VHOST}`;
12 const QUEUE = "order_created";
13
14 const initRabbit = async () => {
15   if (channel) return;
16
17   const conn = await amqp.connect(RABBIT_URL);
18   channel = await conn.createChannel();
19   await channel.assertQueue(QUEUE, { durable: true });
20 };
21
22 const corsMiddleware = cors({
23   origin: process.env.ALLOWED_ORIGINS?.split(",") || [
24     "https://atelier6-rmq-frontend.vercel.app",
25     "http://localhost:3000",
26     "http://localhost:5173",
27   ],
28   credentials: true,
29   methods: ["GET", "POST", "PUT", "DELETE", "OPTIONS"],
30   allowedHeaders: ["Content-Type", "Authorization"],
31 });
32
33 module.exports = async (req, res) => {
34   await new Promise((resolve) => corsMiddleware(req, res, resolve));
35
36   // Handle preflight requests
37   if (req.method === "OPTIONS") {
```

```
38     return res.status(200).end();
39   }
40
41   if (req.method !== "POST") {
42     return res.status(405).json({ error: "Method not allowed" });
43   }
44
45   try {
```

5- Service 2 : Email Worker (Consommateur)

<https://email-worker-atelier6-render.onrender.com>

Logs :

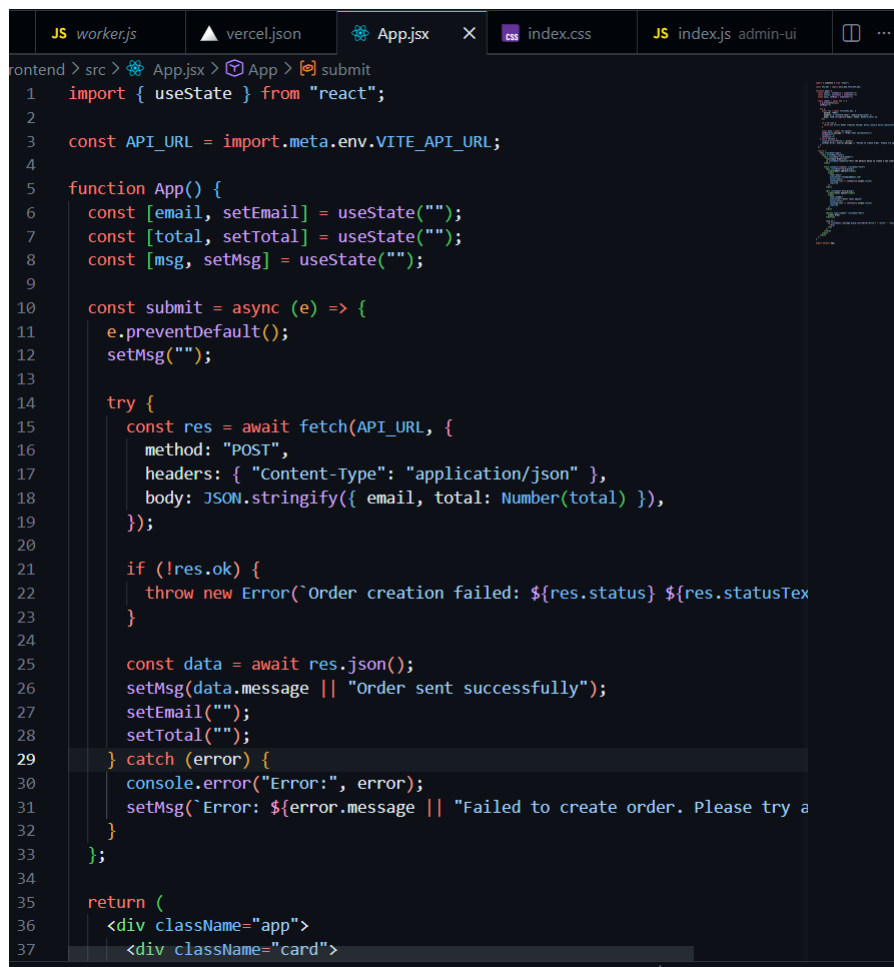
```
Mar 3
11:07:57 AM [qq2dm] 📧 Email Worker waiting for messages...
11:46:21 AM ==> Deploying...
11:46:21 AM ==> Setting WEB_CONCURRENCY=1 by default, based on available CPUs in the instance
11:47:49 AM [dsx46] ==> Running 'node worker.js'
11:47:50 AM [dsx46] 📧 Email Worker waiting for messages...
11:47:50 AM [dsx46] 📧 Email sent to: mohamedsaidbenyaiche@gmail.com
11:47:51 AM [dsx46] 📧 Email sent to: mohamedsaidbenyaiche@gmail.com
11:47:53 AM [dsx46] 📧 Email sent to: mohamedsaidbenyaiche@gmail.com
11:47:54 AM [dsx46] 📧 Email sent to: mohamedsaidbenyaiche@gmail.com
11:47:56 AM [dsx46] 📧 Email sent to: mikey@gmail.com
11:51:12 AM ==> Docs on specifying a port: https://render.com/docs/web-services#port-binding
11:52:15 AM ==> No open ports detected, continuing to scan...
11:52:15 AM ==> Docs on specifying a port: https://render.com/docs/web-services#port-binding
11:52:47 AM ==> Port scan timeout reached, no open ports detected. Bind your service to at least one port. If you don't need to receive traffic on any port, create a background worker instead.
11:52:47 AM ==> Docs on specifying a port: https://render.com/docs/web-services#port-binding
12:01:22 PM ==> Timed Out
```

Render peut exécuter un email-worker en arrière-plan, mais avec le plan gratuit le service se met en veille, donc il faut le relancer manuellement.

```
pi JS worker.js x vercel.json App.jsx index.css JS index.js admin-ui
email-worker > JS worker.js > ...
1  const amqp = require("amqplib");
2
3  const USERNAME = process.env.RABBIT_USER;
4  const PASSWORD = process.env.RABBIT_PASS;
5  const HOST = process.env.RABBIT_HOST;
6  const VHOST = process.env.RABBIT_VHOST;
7
8  const RABBIT_URL = `amqp://${USERNAME}:${PASSWORD}@${HOST}:5672/${VHOST}`;
9  const QUEUE = "order_created";
10
11  async function startWorker() {
12    try {
13      const conn = await amqp.connect(RABBIT_URL);
14      const ch = await conn.createChannel();
15
16      await ch.assertQueue(QUEUE, { durable: true });
17      ch.prefetch(1);
18
19      console.log("📧 Email Worker waiting for messages...");
20
21      ch.consume(QUEUE, async (msg) => {
22        if (!msg) return;
23
24        try {
25          const order = JSON.parse(msg.content.toString());
26          console.log("📧 Email sent to:", order.email);
27
28          // simulate email sending
29          await new Promise((r) => setTimeout(r, 1500));
30
31          ch.ack(msg);
32        } catch (err) {
33          console.error("❌ Message processing failed:", err.message);
34
35          // requeue message
36          ch.nack(msg, false, true);
37        }
38      });
39    }
40  }
```

6- Frontend (React – déployé sur Vercel)

<https://atelier6-rmq-frontend.vercel.app/>



```
JS workenjs  ▲ vercel.json  App.jsx  X  index.css  JS index.js admin-ui  ...
frontend > src > App.jsx > App > submit
1  import { useState } from "react";
2
3  const API_URL = import.meta.env.VITE_API_URL;
4
5  function App() {
6    const [email, setEmail] = useState("");
7    const [total, setTotal] = useState("");
8    const [msg, setMsg] = useState("");
9
10   const submit = async (e) => {
11     e.preventDefault();
12     setMsg("");
13
14     try {
15       const res = await fetch(API_URL, {
16         method: "POST",
17         headers: { "Content-Type": "application/json" },
18         body: JSON.stringify({ email, total: Number(total) }),
19       });
20
21       if (!res.ok) {
22         throw new Error(`Order creation failed: ${res.status} ${res.statusText}`);
23       }
24
25       const data = await res.json();
26       setMsg(data.message || "Order sent successfully");
27       setEmail("");
28       setTotal("");
29     } catch (error) {
30       console.error("Error:", error);
31       setMsg(`Error: ${error.message} || "Failed to create order. Please try a`);
32     }
33   };
34
35   return (
36     <div className="app">
37       <div className="card">
```